

SIMCRA

Sistema Integrado de Monitoreo de Recursos y Contingencia Ambiental

Eva Gallardo Romero

27 de octubre de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Carga e introducción . . . 3
3.2 Inicio . . . 3
3.3 Alertas y detalle . . . 3
3.4 Desaparecidos . . . 3
3.5 Encuesta y estadísticas . . . 3

4 Requisitos del sistema **4**
4.1 Requisitos funcionales . . . 4
4.2 Requisitos no funcionales . . . 4

5 Documentación técnica **4**
5.1 LandingController.java . . . 4
5.2 AlertasManager.java . . . 6
5.3 AlertaReceiver.java . . . 7
5.4 DesaparecidosManager.java . . . 8
5.5 General.java . . . 8

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: SIMCRA.

La aplicación instalable (APK) está en la sección Releases, el diseño en Figma y la documentación del código generada con Javadoc, en evagr11.github.io/SIMCRA/javadoc.

1 Descripción general

SIMCRA (*Sistema Integrado de Monitoreo de Recursos y Contingencia Ambiental*) es una app **Android** en **Java** que simula ser el único sistema de información que sigue funcionando durante una catástrofe en España: una sequía extrema que lleva al racionamiento, a los apagones y a una crisis sanitaria. Es una **historia de ficción**: los hechos, las cifras y las personas son inventados.

La historia **avanza en tiempo real durante 7 días** a partir del primer día que se abre la app:

- **33 alertas** con titular, resumen y texto ampliado. Llegan como **notificaciones** a su hora (08:00, 12:30, 16:00, 18:00 y 21:00) aunque la app esté cerrada, y en la app solo se ven las que ya han llegado.
- Un **registro de 129 desaparecidos** que se va desbloqueando con las alertas de reportes de cada día.
- La pantalla de inicio muestra la fecha, la **batería** en tiempo real, el modelo y la versión de Android, y los valores del **giroscopio** y el **acelerómetro**.
- Una **encuesta emocional** diaria antes de ver las estadísticas de agua, energía, comida y moral (valores simulados).
- La primera vez, dos pantallas cuentan qué es SIMCRA y su red satelital ficticia.

Es un proyecto de **Programación Multimedia y Dispositivos Móviles** de 2.º de DAM para practicar notificaciones, alarmas exactas, `BroadcastReceiver`, sensores, listas y ficheros. Esta es la segunda versión, en la que la lógica de inicio se separó de la vista en un `LandingController`.

2 Diagrama de clases

El código sigue el patrón **Modelo / Vista / Controlador**. Los controladores generan y leen los ficheros de alertas y desaparecidos y programan las alarmas; las vistas heredan de `BaseActivity`, que gestiona la barra de navegación inferior.

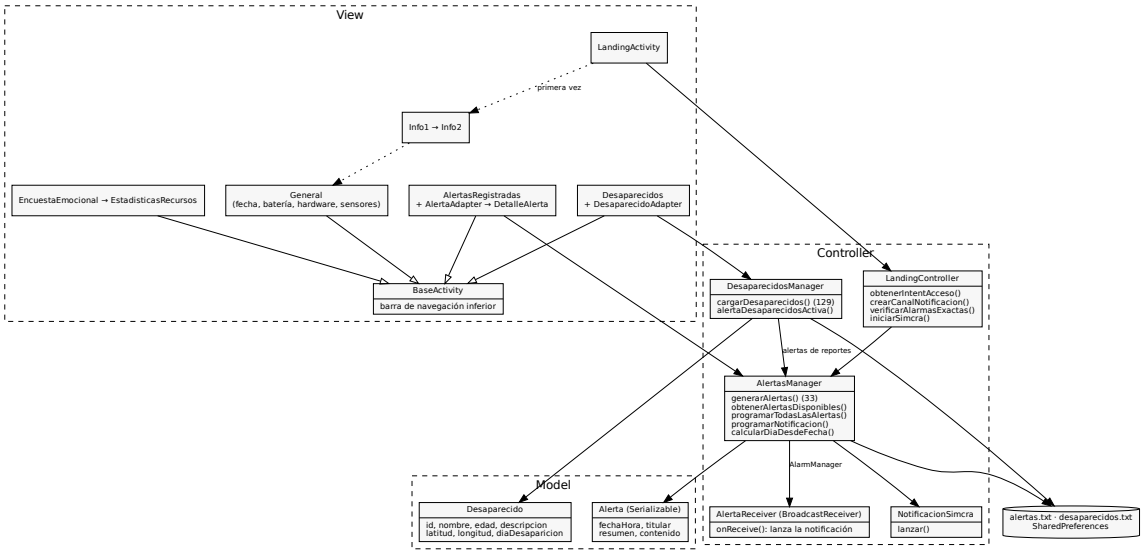


Figura 1: Diagrama de clases de SIMCRA

3 Diseño

El diseño se hizo primero en **Figma**: pantallas de móvil con una cabecera con el logo de SIMCRA y el título, el contenido en tarjetas redondeadas y una barra inferior con cuatro iconos (Inicio, Alertas, Desaparecidos y Estadísticas).

3.1 Carga e introducción

El logo con el nombre y un indicador de carga, y después el botón *Accede a tu información*. La primera vez, dos pantallas de texto explican el papel del usuario (analista de riesgos del Comité Técnico Nacional) y cómo se reciben los datos.

3.2 Inicio

Saluda con la fecha («Hoy es…») y enseña la información del dispositivo: batería, fabricante, modelo, versión de Android y los valores de los sensores.

3.3 Alertas y detalle

La lista de alertas recibidas con su titular y resumen; al pulsar una se abre su texto completo.

3.4 Desaparecidos

Fichas con el nombre, la edad y la última ubicación de cada persona.

3.5 Encuesta y estadísticas

La pregunta *¿Cómo te sientes?* con un deslizador entre una cara triste y una feliz. Al enviarla se ven las barras de agua, energía, comida y moral.

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La primera vez, la app debe mostrar la introducción; las siguientes, ir directa al inicio.
RF02	Debe pedir el permiso de **notificaciones** (Android 13+) y el de **alarmas exactas** (Android 12+).
RF03	Al empezar, debe guardar la **fecha de inicio**, generar las 33 alertas de los 7 días y **programarlas** todas.
RF04	Cada alerta debe llegar como **notificación** a su hora aunque la app esté cerrada.
RF05	En la lista solo deben verse las alertas cuya fecha y hora ya han pasado.
RF06	Al pulsar una alerta se debe abrir su **texto completo**.
RF07	Los desaparecidos de cada día deben verse cuando llega su alerta de reportes.
RF08	El inicio debe mostrar la fecha, la **batería** en tiempo real, los datos del dispositivo y los sensores.
RF09	Las estadísticas solo se ven después de enviar la **encuesta** del día.
RF10	La barra inferior debe llevar a Inicio, Alertas, Desaparecidos y Estadísticas.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	App **Android nativa** en **Java 11** con vistas XML (minSdk 24, targetSdk 36).
RNF02	Arquitectura **Modelo / Vista / Controlador**.
RNF03	Los datos se guardan en **ficheros** del almacenamiento interno y en `SharedPreferences`, sin internet.
RNF04	Las notificaciones usan un **canal** propio de prioridad alta.
RNF05	Si falta un permiso, la app no se cierra: simplemente no lanza la notificación.
RNF06	El código debe tener comentarios **Javadoc**.

5 Documentación técnica

Este anexo agrupa la documentación por clases, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `import`, los `findViewById` y los comentarios Javadoc.

5.1 LandingController.java

5.1.1 Ubicación

`com.example.simcra2.Controller.LandingController`

Descripción general:
La lógica de la pantalla de inicio: a dónde ir, el canal de notificaciones, los permisos y el arranque de la historia.

5.1.2 Funciones

obtenerIntentAcceso **Descripción:** La primera vez lleva a la introducción y lo apunta; después, al inicio.

Código comentado:

```
public Intent obtenerIntentAcceso(Context context) {
    SharedPreferences prefs = context.getSharedPreferences(PREFS_APP, Context.
        MODE_PRIVATE);
    boolean esPrimeraVez = prefs.getBoolean(KEY_PRIMERA_VEZ, true);
    if (esPrimeraVez) {
        prefs.edit().putBoolean(KEY_PRIMERA_VEZ, false).apply();
        return new Intent(context, Info1.class);
    } else {
        return new Intent(context, General.class);
    }
}
```

crearCanalNotificacion **Descripción:** Crea el canal `canal_alertas` de importancia alta (Android 8+).

Código comentado:

```
public void crearCanalNotificacion(Context context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel canal = new NotificationChannel(
                "canal_alertas",
                "Alertas Simcra",
                NotificationManager.IMPORTANCE_HIGH
        );
        canal.setDescription("Notificaciones importantes de alertas");
        NotificationManager manager = context.getSystemService(NotificationManager.
            class);
        manager.createNotificationChannel(canal);
    }
}
```

verificarAlarmasExactas e iniciarSimcra **Descripción:** Si no hay permiso de alarmas exactas, abre los ajustes para darlo. Con permiso, empieza la historia una sola vez: guarda la fecha de inicio, genera las alertas y las programa.

Código comentado:

```
public void verificarAlarmasExactas(Context context) {
    if (yaVerificado) return;
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        AlarmManager alarmManager = (AlarmManager) context.getSystemService(Context.
            ALARM_SERVICE);
        if (!alarmManager.canScheduleExactAlarms()) {
            Toast.makeText(context, "Activa las alarmas exactas para recibir alertas
                programadas", Toast.LENGTH_LONG).show();
            Intent intent = new Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
            intent.setData(Uri.parse("package:" + context.getPackageName()));
            context.startActivity(intent);
            return;
        }
    }
    yaVerificado = true;
    iniciarSimcra(context);
}

public void iniciarSimcra(Context context) {
    SharedPreferences prefs = context.getSharedPreferences(PREFS_APP, Context.
        MODE_PRIVATE);
    if (prefs.contains("fecha_inicio_alertas")) return;      // ya empezó: no se repite
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        prefs.edit().putString("fecha_inicio_alertas", java.time.LocalDate.now().
            toString()).apply();
    }
    AlertasManager.generarAlertas(context);
    AlertasManager.programarTodasLasAlertas(context);
```

```
}
```

5.2 AlertasManager.java

5.2.1 Ubicación

`com.example.simcra2.Controller.AlertasManager`

Descripción general:
Las 33 alertas: las escribe en `alertas.txt` con fechas a partir de hoy, decide cuáles se ven y las programa como alarmas.

5.2.2 Funciones

generarAlertas **Descripción:** Cada día es una tabla con la hora, el titular, el resumen y el texto. Se escriben en el fichero como `fecha|titular|resumen|contenido`. Se muestra el inicio del día 1.

Código comentado:

```
FileOutputStream fos = context.openFileOutput("alertas.txt", Context.MODE_PRIVATE);
LocalDate base = LocalDate.now();                    // el día 1 es hoy

String[][] alertasDia1 = new String[][] {
        {"08:00", "Nivel de Embalses al 20%.", "Se dispara la inflación de alimentos b
            ásicos.",
                "La Confederación Hidrográfica del Tajo (CHT) confirma ..."},
        {"12:30", "Comité de Ministros. Mensaje de Calma.", "Sanción al acaparamiento.
            ", "..."},
        {"18:00", "Apagones Programados", "en el interior de Teruel y Cuenca. ...", "
            ..."},
        {"21:00", "Fichas Abiertas: 5 Reportes.", "Huida preventiva de Madrid.", "..."
            },
};
// ... días 2 a 7; cada línea se escribe con la fecha base + (día - 1)
```

obtenerAlertasDisponibles **Descripción:** Lee el fichero y devuelve solo las alertas cuya fecha y hora ya han pasado. En la versión con modo desarrollador se pueden adelantar a mano.

Código comentado:

```
try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(context.openFileInput("alertas.txt")))) {
    String linea;
    while ((linea = reader.readLine()) != null) {
        String[] partes = linea.split("\\|");
        if (partes.length == 4) {
            LocalDateTime fechaProgramada = LocalDateTime.parse(partes[0], formatter);
            boolean desbloqueada = false;
            if (MODO_DESARROLLADOR) {
                // ... las adelantadas a mano también cuentan
            } else {
                if (fechaProgramada.isBefore(ahora) || fechaProgramada.isEqual(ahora))
                    {
                    desbloqueada = true;
                }
            }
            if (desbloqueada) {
                lista.add(new Alerta(partes[0], partes[1], partes[2], partes[3]));
            }
            contador++;
        }
    }
}
```

programarNotificacion **Descripción:** Programa una alarma exacta que despierta al móvil aunque esté en reposo. Cada alarma usa su hora como código para no pisar a las demás.

Código comentado:

```
public static void programarNotificacion(Context context, String titulo, String
    mensaje, long tiempoEnMillis) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        AlarmManager alarmManager = (AlarmManager) context.getSystemService(Context.
            ALARM_SERVICE);
        if (!alarmManager.canScheduleExactAlarms()) return;
    }
    Intent intent = new Intent(context, AlertaReceiver.class);
    intent.putExtra("titulo", titulo);
    intent.putExtra("mensaje", mensaje);
    PendingIntent pendingIntent = PendingIntent.getBroadcast(
            context,
            (int) tiempoEnMillis,                    // código distinto por alarma
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
    );
    AlarmManager alarmManager = (AlarmManager) context.getSystemService(Context.
        ALARM_SERVICE);
    alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, tiempoEnMillis,
        pendingIntent);
}
```

calcularDiaDesdeFecha **Descripción:** Calcula en qué día de la historia cae una alerta contando desde la fecha de inicio.

Código comentado:

```
Date fechaAlerta = formatter.parse(fechaStr);
String fechaInicioStr = prefs.getString("fecha_inicio_alertas", null);
if (fechaInicioStr == null) return 1;
Date fechaInicio = dateOnly.parse(fechaInicioStr);
long diffMillis = fechaAlerta.getTime() - fechaInicio.getTime();
long diffDias = diffMillis / (1000 * 60 * 60 * 24);
return (int) diffDias + 1;                                  // el primer día es el 1
```

5.3 AlertaReceiver.java

5.3.1 Ubicación

`com.example.simcra2.Controller.AlertaReceiver`

Descripción general:
`BroadcastReceiver` que recibe la alarma y lanza la notificación.

5.3.2 Funciones

onReceive **Descripción:** Construye la notificación con el título y el mensaje de la alarma y la muestra si hay permiso.

Código comentado:

```
@Override
public void onReceive(Context context, Intent intent) {
    String titulo = intent.getStringExtra("titulo");
    String mensaje = intent.getStringExtra("mensaje");
    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, "
        canal_alertas")
            .setSmallIcon(R.mipmap.ic_alert)
            .setContentTitle(titulo)
            .setContentText(mensaje)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true);
    try {
        if (android.os.Build.VERSION.SDK_INT < android.os.Build.VERSION_CODES.TIRAMISU
             ||
                context.checkSelfPermission(android.Manifest.permission.
                    POST_NOTIFICATIONS)
```

```
                        == PackageManager.PERMISSION_GRANTED) {
                NotificationManagerCompat.from(context).notify((int) System.
                    currentTimeMillis(), builder.build());
            }
        } catch (SecurityException e) {
            e.printStackTrace();                            // sin permiso, la app no se
                cierra
        }
    }
```

5.4 DesaparecidosManager.java

5.4.1 Ubicación

`com.example.simcra2.Controller.DesaparecidosManager`

Descripción general:
Las 129 fichas de desaparecidos y cuáles se pueden ver.

5.4.2 Funciones

cargarDesaparecidos **Descripción:** Crea el fichero la primera vez, lo lee y devuelve las fichas hasta el último día cuya alerta de reportes ya ha llegado.

Código comentado:

```
File archivo = new File(context.getFilesDir(), ARCHIVO);
if (!archivo.exists()) {
    crearArchivoInicial(archivo);
}

List<Desaparecido> todos = new ArrayList<>();
try (BufferedReader reader = new BufferedReader(new FileReader(archivo))) {
    String linea;
    while ((linea = reader.readLine()) != null) {
        String[] partes = linea.split("\\|");          // id|nombre|edad|descripción|
            lat|lon|día
        if (partes.length == 7) {
            todos.add(new Desaparecido(Integer.parseInt(partes[0]), partes[1],
                    Integer.parseInt(partes[2]), partes[3], Double.parseDouble(partes
                        [4]),
                    Double.parseDouble(partes[5]), Integer.parseInt(partes[6])));
        }
    }
} catch (IOException e) {
    e.printStackTrace();
}

if (!alertaDesaparecidosActiva(context)) {
    return new ArrayList<>();                           // aún no ha llegado ningún
        reporte
}

int diaActual = Math.max(prefs.getInt(CLAVE_DIA, 1), diaDeReportesDesbloqueado(context
    ));
List<Desaparecido> visibles = new ArrayList<>();
for (Desaparecido d : todos) {
    if (d.diaDesaparicion <= diaActual) {
        visibles.add(d);
    }
}
return visibles;
```

5.5 General.java

5.5.1 Ubicación

`com.example.simcra2.View.General`

Descripción general:
La pantalla de inicio con los datos del dispositivo.

5.5.2 Funciones

Bateria **Descripción:** Escucha el aviso del sistema `ACTION_BATTERY_CHANGED` y escribe el porcentaje.

Código comentado:

```
public void Bateria(){
    batteryReceiver = new BroadcastReceiver() {
        @Override
        public void onReceive(Context context, Intent intent) {
            int level = intent.getIntExtra(BatteryManager.EXTRA_LEVEL, -1);
            int scale = intent.getIntExtra(BatteryManager.EXTRA_SCALE, -1);
            int batteryPct = (int) ((level / (float) scale) * 100);
            txtBateria.setText(batteryPct + "%");
        }
    };
    registerReceiver(batteryReceiver, new IntentFilter(Intent.ACTION_BATTERY_CHANGED))
        ;
}
```

Sensores **Descripción:** Registra un mismo *listener* para el giroscopio y el acelerómetro y escribe sus tres ejes. Se da de baja en `onPause` y se vuelve a registrar en `onResume`.

Código comentado:

```
public void Sensores(){
    sensorManager = (SensorManager) getSystemService(SENSOR_SERVICE);
    Sensor sensorGiroscopio = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
    Sensor sensorAcelerometro = sensorManager.getDefaultSensor(Sensor.
        TYPE_ACCELEROMETER);
    sensorListener = new SensorEventListener() {
        @Override
        public void onSensorChanged(SensorEvent event) {
            if (event.sensor.getType() == Sensor.TYPE_GYROSCOPE) {
                txtGiroscopio.setText(String.format("X: %.2f\nY: %.2f\nZ: %.2f", event
                    .values[0], event.values[1], event.values[2]));
            } else if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
                txtAcelerometro.setText(String.format("X: %.2f\nY: %.2f\nZ: %.2f",
                    event.values[0], event.values[1], event.values[2]));
            }
        }
        @Override
        public void onAccuracyChanged(Sensor sensor, int accuracy) {}
    };
    sensorManager.registerListener(sensorListener, sensorGiroscopio, SensorManager.
        SENSOR_DELAY_UI);
    sensorManager.registerListener(sensorListener, sensorAcelerometro, SensorManager.
        SENSOR_DELAY_UI);
}
```